GhostHouses - Stage A

Repository: https://github.com/bbdaria/GhostHouses

1. User Stories (HLD vs Current)

The original HLD defined 16 user stories. We now track 17 User Story issues: the original 16 plus one added after HLD (US-17 export buildings table to Excel for backup/restore, Issue #76).

Canceled per requirement change: US-14 Delete log entries (logs are immutable) and US-13 Add/edit log entries (logs are auto-generated and immutable).

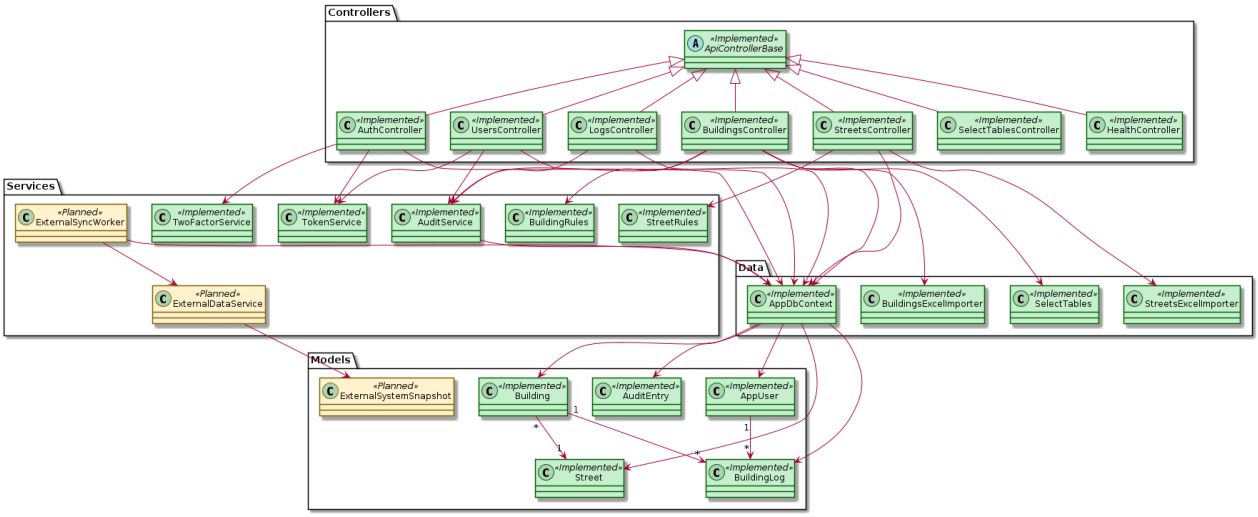
Additional scope since HLD: multi-select building card export (Issue #74) and expanded import/export workflows + template converter.

Current status: core workflows are Done. Backlog items for Stage B are US-2 2FA integration, US-3 admin permission management, US-7 building-card template update, and US-16 external system sync.

US ID	Description	Status	Notes
US-1	Clear role model (Viewer/Editor/Admin)	Implemented	Issue #20 scheduled to close before presentation.
US-2	2FA login	In Progress	Issue #21 open – OTP flow exists, real OTP integration pending (#4).
US-3	User permissions control	In Progress	Issue #22 open – role gates exist, final admin controls pending.
US-4	View building details	Implemented	Issue #23 closed.
US-5	Search & filter buildings	Implemented	Issue #24 closed.
US-6	View building card with photos	Implemented	Issue #25 closed.
US-7	Export building card for presentations	Mostly Implemented	Issue #26 open only for template updates (#82).
US-8	Add new buildings	Implemented	Issue #27 closed.
US-9	Update existing buildings	Implemented	Issue #28 closed.
US-10	Remove buildings with safeguards	Implemented	Issue #29 closed.
US-11	View building change log	Implemented	Issue #30 closed.
US-12	Search & filter logs	Implemented	Issue #31 closed.
US-13	Add/edit log entries	Implemented	Issue #32 closed (logs auto-generated from changes).
US-14	Delete log entries	Removed	Requirement removed, logs are immutable (Issue #73).

US-15	Full audit trail	Implemented	Issue #34 closed.
US-16	External system sync	Planned	Issue #35 open (#5, #78).
US-17	Export buildings table to Excel for backup/restore	Implemented	Added after HLD, Issue #76 closed.

2. Design / Class Diagram



Layered architecture: Controllers → Services → Data → Models. Planned components (external sync) are marked in yellow, implemented components are green.

Arrow legend: A → B means A uses/depends on B, A <|– B means inheritance, 1→* indicates one-to-many relationships. Incoming arrows mean others depend on this component, outgoing arrows mean it depends on others.

Layer definitions:

- DTOs (Data Transfer Objects): small request/response shapes so we don't expose database entities directly.
- Controllers: API entry points that validate input, call services, and return DTOs.
- Services: business logic (validation, import/export workflows, logging, integrations like OTP/external sync).
- Data layer: EF Core DbContext, migrations, and import utilities, uses LINQ (C# query syntax) instead of raw SQL.
- Models: domain entities (Building, Street, User, Log) plus metadata like FieldSpecAttribute.

3. GitHub Repository & Handoff Readiness

Repository is structured to support handoff and maintenance:

- Clear separation of backend/frontend (`project/web-server`).
- Documentation stored under `docs/` (HLD, architecture, conventions, submissions).
- Issue workflow with branch naming `Issues/#<number>-<slug>` and one issue per branch.
- Issue guard workflow comments on missing metadata (milestone, labels, parent story) and rule violations.
- Tests live under `tests/` (currently minimal smoke tests).
- Issue guard runs on issue events and daily (scheduled) to catch drift automatically.
- Issue guard checks project status, parent story, and branch linkage in addition to labels/milestones.

4. CI/CD and Deployment Flow

Current state:

- Application is built and run locally via Docker Compose.
- Temporary client testing host: team PC kept online with No-IP + port-forwarding (external HTTPS 443 → frontend 443, backend internal 8080, PostgreSQL 5432, pgAdmin HTTPS 8443). Client access: https://ghosthouses.myddns.me/.

Current port layout: Frontend HTTPS 443 (public), backend internal 8080 (not exposed), PostgreSQL 5432 (internal), pgAdmin HTTPS 8443 (admin-only).

- Network isolation: app-net (frontend↔backend), db-net (backend↔db), admin-net (pgAdmin↔db).
- Issue-guard workflow exists for metadata enforcement.

Planned next stage:

- Add CI pipeline for build + tests on PR/merge.
- Deploy to client Windows Server with HTTPS once IT provides the environment (Issue #78).

HLD vs current:

- Same as HLD: GitHub workflow (dev for ongoing work, main for stable releases) and a Dockerized stack.
- Changed: official Windows Server deployment is delayed, a temporary host is used to keep client testing active.

5. POC (from HLD)

HLD POC: validate Excel ingestion of municipal data into the system database.

What we delivered beyond the POC:

- Single source of truth for Buildings/Streets rules shared by add/edit/import/export.
- Immutable logs (audit-grade).
- New strict template is the only accepted import format.
- Temporary converter maps legacy files to the new template, validation is enforced on import.
- Import uses staged validation and conflict resolution (missing required fields first, then duplicates).

Outcome:

- Reliable, production-grade import with staged validation and conflict resolution.

6. Current Project Risk

Primary risk: deployment on the client's Windows Server with unknown infrastructure and external system integrations. This affects OTP, API connectivity, and production configuration. Mitigation: early coordination with client IT, containerized deployment plan, and API contract definition.

Short-term mitigation includes mock adapters for external APIs and temporary client testing (No-IP + port-forwarding) until IT provides the official Windows Server environment.

7. Risk Management Table

Project Risk Severity	Likelihood of Risk	Mitigation	Severity After Mitigation	Likelihood After Mitigation
High	Medium	Define deployment plan with client IT, test pilot deployment	Medium	Low
High	Medium	Define external API contracts, build mocks/adapters	Medium	Low
Medium	High	Integrate OTP with client provider, maintain fallback	Medium	Low
Medium	High	Staged import validation + conflict resolution	Low	Medium
Medium	Medium	Docs, conventions, issue guard, structured repo	Low	Low

8. Current Backlog

All Stage A milestone issues and Current Sprint items are closed (Done or Canceled).

All remaining open items are in Stage B.
Backlog should be reviewed in the GitHub Project board (visual view).

Backlog (open issues):

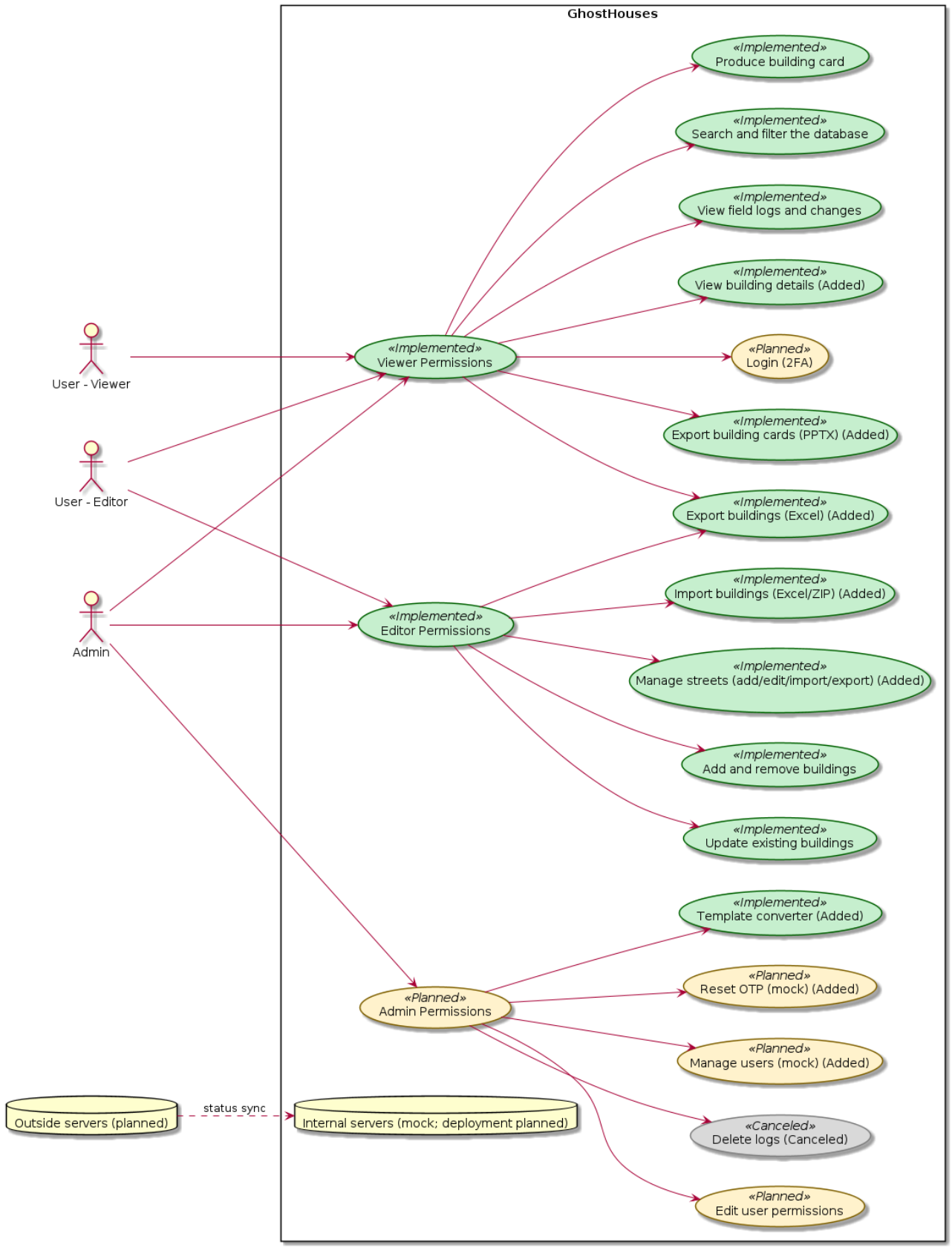
- #4 Real OTP integration
- #5 External data sources integration
- #78 Deployment on client Windows Server
- #82 Building card template update (awaiting client)

Open User Stories tied to backlog:

- US-2 2FA login
- US-3 Permissions
- US-7 Building card export (template update)
- US-16 External system sync

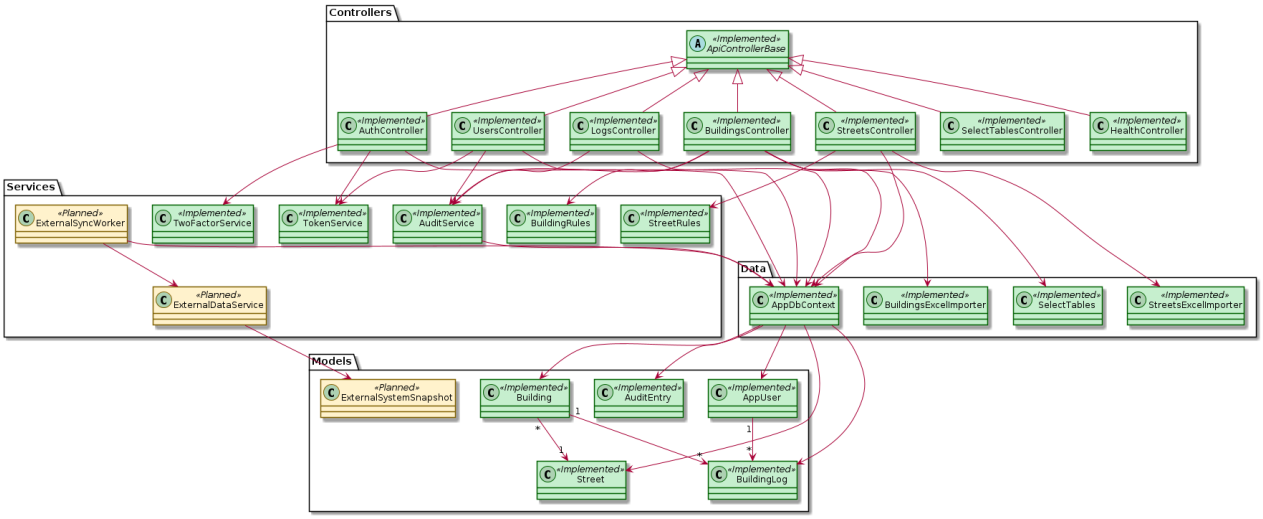
Why Stage B: these depend on external inputs (client IT deployment access, external municipal APIs, OTP provider integration, and a new building card template), so they cannot be completed in Stage A.

9. UML: Use Case (Implemented / Planned / Backlog)



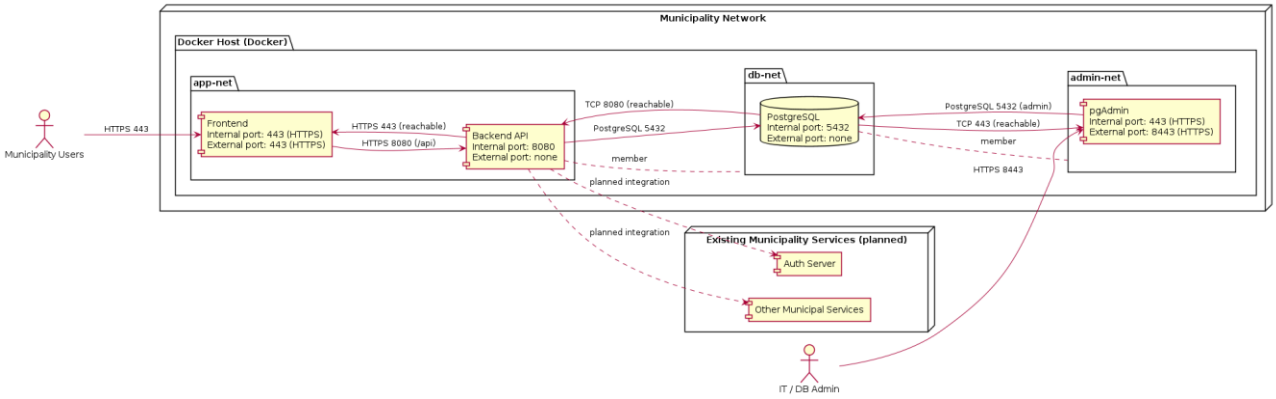
Color legend: Green = implemented, Yellow = planned, Red = backlog, Gray = canceled/removed. Added items are labeled (Added).

10. UML: Class Diagram



Implementation-accurate class diagram, planned components are marked separately.

11. UML: Deployment (Desired / Target)

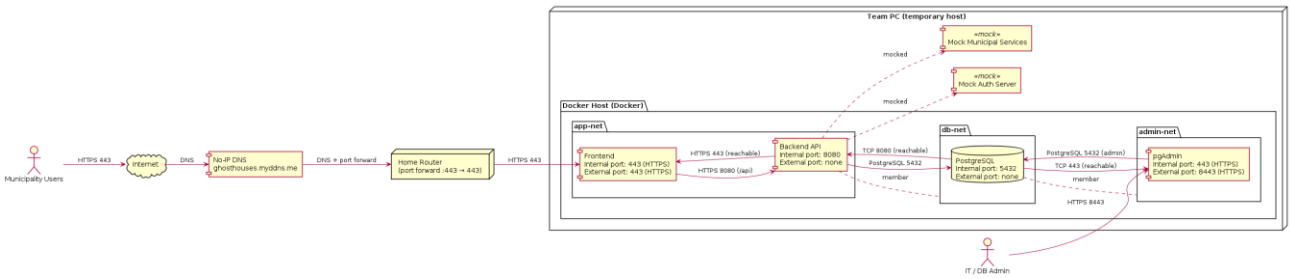


Target deployment: municipality users access the Frontend over HTTPS, Frontend calls Backend, Backend connects to PostgreSQL, Auth Server, and municipal services inside the network.

Docker containers: frontend + backend run in containers on the web server, PostgreSQL + pgAdmin run in containers on the database server.

pgAdmin is for municipality IT/DB admins only (not end users).

12. UML: Deployment (Current / Existing)



Current temporary hosting on team PC with No-IP DNS (https://ghosthouses.myddns.me) and port-forwarding (external HTTPS 443 → frontend 443, backend API internal 8080, PostgreSQL 5432, pgAdmin HTTPS 8443). Network split mirrors the desired setup: app-net (frontend ↔ backend), db-net (backend ↔ DB), admin-net (pgAdmin ↔ DB).